

ASSIGNMENT 2 – NATURAL LANGUAGE UNDERSTANDING

Name: Yesha Shah
Roll Number: B22CS067

Overview

This assignment focuses on two core Natural Language Processing (NLP) tasks. The first involves constructing a high-quality text corpus from real-world institutional documents and performing preprocessing and analysis. The second task explores sequence modeling by generating Indian names using a character-level Recurrent Neural Network (RNN). Together, these tasks demonstrate practical skills in data collection, text preprocessing, and basic neural language modeling.

PROBLEM 1: CORPUS CREATION AND PROCESSING

Objective

The goal of this task was to collect textual data from multiple IIT Jodhpur sources and construct a clean, structured corpus suitable for NLP tasks such as tokenization, frequency analysis, and potential embedding generation.

Data Collection

The dataset was created using multiple official IIT Jodhpur documents to ensure diversity and authenticity. These included:

- Academic curriculum documents (B.Tech and M.Tech)
- Course details and specialization descriptions
- Academic regulations and policy documents
- Board of Governors meeting minutes
- Student council documentation

This combination ensured that the corpus contained both structured and unstructured academic text.

Methodology

The corpus creation pipeline was implemented step-by-step as follows:

- Step 1: Extracted text from PDF documents using [pdfplumber](#)
- Step 2: Converted all extracted text to lowercase for uniformity
- Step 3: Removed numbers and punctuation using regular expressions
- Step 4: Tokenized the cleaned text into individual words using NLTK
- Step 5: Removed stopwords (common words like “the”, “is”, etc.)
- Step 6: Performed frequency analysis using word counting

Additionally, minor adjustments such as filtering noisy tokens and handling extraction inconsistencies were made to improve corpus quality.

Results and Observations

- The corpus contained a large volume of text (~400k+ characters), indicating strong data coverage
- The vocabulary was diverse, covering academic, administrative, and technical terms
- Frequently occurring words included:
 - *course, student, program, credit, semester, academic*

These results indicate that the corpus successfully captures the structure and language of institutional academic content.

```
[15] ✓ Os  from collections import Counter

word_freq = Counter(filtered_tokens)

print("Top 10 Words:")
top10 = word_freq.most_common(10)

for word, freq in top10:
    print(word, ":", freq)

... Top 10 Words:
lectures : 674
data : 466
learning : 447
computer : 362
course : 355
design : 303
systems : 290
algorithms : 284
tech : 281
engineering : 270
```

Top 10 most frequent words (from your notebook output)

Conclusion (Problem 1)

The corpus created is clean, diverse, and suitable for downstream NLP tasks. The use of multiple real-world documents enhances its reliability and representational quality.

PROBLEM 2: NAME GENERATION USING RNN

Objective

The goal of this task was to build a character-level sequence model capable of learning patterns in Indian names and generating new, similar names.

Approach

A Vanilla Recurrent Neural Network (RNN) was implemented to model sequential dependencies between characters. The model learns to predict the next character in a sequence based on previous characters.

Model Details

- Model Type: Vanilla RNN
- Input: Character sequences from Indian names
- Hidden Size: 64
- Epochs: 5
- Optimizer: Adam
- Loss Function: CrossEntropyLoss

Due to time constraints, a simpler RNN was chosen instead of more advanced architectures such as BiLSTM or Attention-based models.

Results

After training, the model was able to generate multiple new names. Some examples include:

- yash sanjas
- ruvi kothar
- ashrasansh
- karushunal
- vinia dhank

Observations

- The model successfully learned common character patterns such as:
 - “sh”, “an”, “ra”, “ya”
- Some generated names were realistic, while others were slightly noisy
- This behavior is expected for a basic RNN trained for a limited number of epochs

```
print("Raw text length:", len(text))

print("Total tokens:", len(tokens))
print("Filtered tokens:", len(filtered_tokens))

import os
size_mb = os.path.getsize("corpus.txt") / (1024*1024)
print("Corpus size (MB):", round(size_mb, 3))

... Raw text length: 383704
Total tokens: 52986
Filtered tokens: 41389
Corpus size (MB): 0.326
```

Conclusion

The RNN model demonstrates the ability to learn sequential patterns in text data. While not perfect, it generates diverse and partially realistic names, showcasing the effectiveness of sequence modeling.

